DB09

복잡한 Retrieval query에 대해 알아보자.

Nested query

> **SQL SELECT의 WHERE Clause 내에 또 다른 SQL SELECT 문이 존재하는 Query**

`Inner query` **: WHERE 절 내부에 존재하는 SELECT문**

`Outer query` **: 해당 WHERE 절을 갖는 SELECT 문**

Inner query는 Outer query의 Attribute를 직접 참조할 수 있다. 그러나 그 반대는 불가능하다.

여러 겹의 중첩이 가능하다.

- 그러나 중첩이 많을수록 Nested query를 처리하는데 **Performance overhead가 발생**한다.
- 고성능을 요구하는 경우, 사용하지 않는 것이 적합하다.

DB에 존재하는 값들을 가져와 계산하여 조건에 사용하기 위해서 사용한다.

모든 Nested query는 항상 여러 Query로 Unnesting이 가능하다.

Comparison Operators

WHERE 절 내에 SELECT를 사용하면 보통 중복을 허용하는 집합이 반환된다. 해당 집합에 적용되는 연산자를 **Comparison Operator**라고 한다.

- **WHERE** Column_name **Comparison operator** (SELECT ~~) 으로 사용한다.

IN Operator

> **해당 Tuple 집합에 주어진 Value랑 같은 값이 존재하면 TRUE, 아니라면 FALSE를 반환한다.**

= ANY (SOME) Operator

> **주어진 tuple과 집합 내에서 어느 하나라도 같다면 TRUE를 반환한다.**

- **= ANY는 IN과 동일**하다.

>, ≥, <, ≤, <> 모두 ANY와 사용할 수 있다.

- <> = ≠
- 하나의 Value가 위 조건을 만족하도록 하는 Tuple이 집합 내에 하나라도 존재한다면 TRUE를 반환한다.
- ANY를 사용하는 실질적 이유이다.

= ALL Operator

> **하나의 Value와 집합 내의 모든 Tuple이 조건을 만족한다면 TRUE를 반환한다.**

>, ≥, <, ≤, <> 모두 ALL과 사용할 수 있다.

- <> = ≠
- ALL도 이 연산자인 경우에만 상한선, 하한선을 정의하는 등 의미 있게 사용할 수 있다.

Avoiding Ambiguities

Inner query에서 사용하는 Table과 Outer query에서 사용하는 Table의 특정 Column 이름이 동일한 경우, Ambiguities가 발생할 수 있다.

Reference rule

> **Ambiguities가 발생한 상황에 어느 table의 attribute인지 명시가 되지 않은 상태라면 명시되지 않은 attribute는 Innermost nested query의 FROM 절에서 정의된 Table의 Attribute라고 생각하고 처리한다.**

- **Unqualified attribute**이 사용된 것에서 가장 안 쪽에 있는 FROM 절의 Table의 attribute라고 판단한다.
- 가장 안 쪽에 있는 FROM 절의 Table에서 해당 Attribute를 찾을 수 없다면 Outer query의 FROM 절에서 찾는 동작을 반복한다.

Aliasing을 통해 해결할 수 있다.

Correlated nested query

> **Inner query가 Outer query의 Table과 Attribute를 참조하는 qeury**

- 이 경우, Outer query의 **Employee table의 각 tuple 마다 Inner query가 실행**된다.

```
SELECT *
FROM Employee e
WHERE EXISTS (
    SELECT *
    FROM Works w
    WHERE w.emp_id = e.emp_id   -- ← outer query 참조
);
```

EXISTS fucntion

Nested query의 결과로 얻은 집합이 비어있는지 아닌지 판단하는 함수

- TRUE: 하나의 Tuple이라도 존재하는 경우
- FALSE: 하나의 Tuple도 존재하지 않는 경우

Correlated nested query에서 주로 사용된다.

NOT EXISTS

EXISTS function의 반대이다.

UNIQUE function

Nested query의 결과에 중복된 Tuple이 존재하는지 확인하는 함수

- TRUE: 중복된 Tuple이라도 존재하지 않는 경우
- FALSE: 중복된 Tuple도 하나라도 존재하는 않는 경우

유용한 기능

Explicit set

> **Query 안에 값을 직접 나열해서 만든 집합**

- EX) (1, 2, 3)
- 보통 WHERE 절에서 **IN**과 함께 사용된다**.**

IS NULL function

attribute_name IS NULL로 사용되며 Attribute의 값이 NULL인지 아닌지 확인하는 함수

- TRUE: NULL인 경우
- FALSE: NULL이 아닌 경우

NULL 값은 다른 값과 비교 자체가 불가능하다.

- 값을 모르는 것이기 때문에 >, <, <> 등의 연산자 적용이 되지 않는다.
- **서로 다른 NULL 값끼리는 비교가 되지 않기 때문에 같지 않다.**
- NULL과의 비교 결과는 **UNKNOWN**이 되고, WHERE 절은 TRUE인 결과만 통과시키기 때문에 NULL 값이 결과에 포함되지 않는 것이다.

JOIN Operation에서도 **Join attribute의 값으로 NULL 값을 갖는 Tuple 쌍은 결과에서 제외**된다.

NULL 값은 비교 자체가 불가능하기 때문에 `IS NULL` **또는** `IN NOT NULL` **함수를 사용**해야 한다.

Renaming attributes

SELECT attribute_name **AS** alias

- 결과로 된 Table의 attribute name을 수정한다.
- **DB에 저장된 Column 명을 실제로 변경한 것이 아니라 결과 테이블에서 보이는 Column 명만 변경**된 것이다.

Aggregate function

> **여러 Tuple의 집합의 정보를 요약하는 하나의 Tuple을 생성하는 함수**

Built-in aggregate function

- **COUNT, SUM, MAX, MIN, AVG**

Grouping

`Grouping attribute` : Grouping의 기준이 되는 attribute

- Grouping의 기준이 되기 때문에 SELECT 문에도 사용되어야 한다.
  - 어떤 기준으로 묶였는지 확인되어야 하기 때문이다.
- **GROUP BY** clause에 의해 특정 된다.
- Grouping attribute가 같은 값을 갖는 Tuple들이 하나의 Group으로 묶이게 된다.

GROUP BY clause가 존재할 때, Aggregate function을 적용하면 **각 Group별로 Aggregate function을 적용한 결과**가 나온다.

SELECT 절에 **Non-aggregate function이면서 Grouping attribute가 아닌 attribute는 사용할 수 없다.**

- 어떤 값을 사용할 지 정해지지 않기 때문이다.

HAVING clause

GROUP BY로 구별된 GROUP 중 특정 GROUP을 선택하는 조건을 명시하는 절.

- 보통 **Aggregate function (attribute_name) operator value** 형태를 사용한다.
- HAVING에 쓰이는 non-aggregate attribute는 반드시 Grouping attributes에 포함되어야 한다.

GROUP BY와 연결되어 사용하는 것이 일반적이다.

- GROUP BY 없이 사용해도 문제 없지만 이런 경우 테이블 전체를 하나의 Group으로 보고 진행하기에 큰 의미가 없다.
    - 하지만, 이 경우에는 반드시 HAVING에 Aggregate function만이 조건으로 사용될 수 있다.

WHERE는 Tuple을 고르는 조건을 명시하는 반면, **HAVING**은 Group을 고르는 조건을 명시한다.

Substrting Pattern Matching

LIKE comparison operator

- 문자열에 대해서만 동작하고 문자열의 패턴을 비교한다.

WHERE attribute_name **LIKE** 패턴으로 사용된다.

- 패턴에는 문자열과 와일드카드가 사용된다.

와일드카드

- **%: 존재하지 않거나 여러 개의 문자가 있을 수 있음을 표현**
- **_: 정확히 하나의 글자가 있음을 표현**

마찬가지로 **LIKE로 NULL을 비교하면 결과는 UNKNOWN이다.**

- WHERE 절에서는 UNKNOWN은 선택되지 않는다.

Arithmetic operators

SQL에서 사용할 수 있는 기본적인 산술 연산자

- **Addition (+), subtraction (-), multiplication (*), and division (/)**

ORDER BY

> **결과로 나오는 Tuple의 순서를 조정할 수 있도록 하는 Clause**

ASC: 오름 차순, Default 값

DESC: 내림차순

EX) **ORDER BY** D.Dname **DESC**, E.Lname **ASC**, E.Fname **ASC**

- D.Dname으로 우선 내림차순 정렬 후, D.Dname이 같은 값이 있다면 E.Lname 기준 오름차순 정렬, 그럼에도 정렬되지 않은 Tuple은 E.Fname으로 **ASC**

Summary

각 Clause의 사용 순서를 정리하면 다음과 같다.

```
SELECT
FROM
WHERE
GROUP BY
HAVING
ORDER BY
```

- 논리적으로는 **FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY** 순으로 실행된다.
- **SELECT, FROM 절만 필수**이고 나머진 선택적으로 사용할 수 있다..